

Movie Fantasy League — App Spec

A web app to run an 8-team movie fantasy auction league, track rosters, and update scoring through the year based on box office and awards data fetched from URLs.

Goals

The app needs to do four things well:

1. **Capture the auction draft** — record which manager bought which film for what price, with sequel/IP tagging baked in.
2. **Display rosters and standings** — show every team's slate, current point totals broken out by source (box office tiers, awards, bonuses), and league-wide leaderboard.
3. **Update scoring from external sources** — let me paste a URL (Box Office Mojo, Variety, Deadline, Academy Awards site, etc.) into a chat-style interface and have Claude parse it, propose score updates, and apply them on confirmation.
4. **Audit log** — every score change is timestamped, attributable to a source URL, and reversible.

This is a private app for ~10 people. No auth needed beyond a shared password gate, no payments, no notifications. Hostable on Vercel + a single Postgres or SQLite instance.

Tech stack

- **Next.js 14+ (App Router)** with TypeScript and Tailwind
- **SQLite via Prisma** for local dev, Postgres for production (Vercel Postgres or Supabase)
- **Anthropic SDK** (`@anthropic-ai/sdk`) for the scoring-update chat interface, using Claude Sonnet 4 with web fetch tool enabled
- **shadcn/ui** for component primitives
- Single shared-password gate via a middleware check against an env var; no per-user accounts

Data model

```

model Manager {
  id          String    @id @default(cuid())
  name        String    @unique
  films       Film[]
  createdAt   DateTime  @default(now())
}

model Film {
  id          String    @id @default(cuid())
  title       String
  releaseDate  DateTime?
  productionBudget Int?  // in millions USD, for sleeper bonus eligibility
  isSequel    Boolean   @default(false)
  isRosterStarter Boolean @default(true)  // false = bench
  manager     Manager   @relation(fields: [managerId], references: [id])
  managerId   String
  auctionPrice Int       // dollars of the $200 budget
  scoreEvents  ScoreEvent[]
  createdAt   DateTime  @default(now())
}

model ScoreEvent {
  id          String    @id @default(cuid())
  film        Film      @relation(fields: [filmId], references: [id])
  filmId      String
  category    String     // "box_office_tier_50M", "oscar_nom_best_picture", "sleeper"
  rawPoints   Float      // before multipliers
  multiplier  Float      @default(1.0)  // 0.75 for sequel BO, 1.25 for original-I
  finalPoints Float      // rawPoints * multiplier
  sourceUrl   String?
  notes       String?
  createdAt   DateTime  @default(now())
  reversedAt  DateTime? // soft delete for audit trail
}

model AuditLog {
  id          String    @id @default(cuid())
  action      String     // "draft_pick", "score_added", "score_reversed", "trade", "trade_cancel"
  payload     Json
  createdAt   DateTime  @default(now())
}

```

Pages and routes

/ — Dashboard

League leaderboard table sorted by total points. Each row links to a team page. Show columns: Manager, Box Office Pts, Awards Pts, Bonus Pts, Total. Below the table, a feed of the last 20 score events across the league.

/draft — Auction entry

A form for entering draft results. Two flows:

- **Bulk paste mode:** A textarea accepting one pick per line in the format `Manager | Film Title | Price | sequel?` — comma or pipe separated. Parse and preview before committing. Useful if I'm running the auction live and entering everything at the end.
- **Single pick mode:** Dropdown for manager, autocomplete/free-text for film title, number input for price, checkbox for "is sequel/IP continuation," optional production budget input. Add to roster on submit.

Show running totals per manager: dollars spent of \$200, films drafted of 8, sequel count of 3.

/team/[managerId] — Team detail

- Manager name, total points, rank
- Roster in a table: Film, Auction Price, Is Sequel, Box Office Pts, Awards Pts, Bonus Pts, Subtotal
- Below: chronological list of all ScoreEvents for this team's films, with source URLs

/scoring — Claude-powered scoring updates

This is the core feature. A chat interface where I paste URLs and ask for updates. Behavior:

- Input box accepts a URL plus optional natural-language instruction ("update box office for all my films" or "score the Oscar nominations")
- On submit, the server route calls the Anthropic API with the web fetch tool enabled and the league's current roster as context
- Claude reads the URL, identifies relevant films (cross-referenced against the roster), and returns a structured proposal: a JSON list of `{filmTitle, category, rawPoints, multiplier, reasoning, sourceUrl}` objects
- The UI renders the proposal as a confirmation table — checkboxes pre-checked for each proposed change, with reasoning shown

- I review, uncheck anything wrong, hit "Apply Selected" → server creates `ScoreEvent` rows for everything checked
- Conversation history persists in the page session so I can iterate ("you missed Wicked, try again")

The system prompt for Claude on this route should include:

- The full scoring rubric (box office tiers, awards weights, multipliers)
- The current roster (every film, its manager, its sequel flag)
- A directive to only propose scoring for films currently on a roster
- A directive to flag uncertainty rather than guess

`/admin` — Manual overrides

- Reverse a `ScoreEvent` (sets `reversedAt`, subtracts from totals)
- Manually add a `ScoreEvent` (for cases Claude can't handle — e.g., a category enum is missing)
- Edit film metadata (toggle sequel flag, update production budget)
- Trade execution: swap film ownership between two managers
- October Swap: drop one film, add a new one (with auction-price set to \$0)

`/rules` — Static page

Render the full scoring rubric for reference. Pull from a constants file so the rules and the scoring engine can never drift.

Scoring engine

A single module `lib/scoring.ts` that owns all rules. The rest of the app calls into it.

```
// Box office tiers — each tier awards points once when the film crosses it
export const BOX_OFFICE_TIERS = [
  { threshold: 50, points: 5 },
  { threshold: 100, points: 10 },
  { threshold: 200, points: 15 },
  { threshold: 300, points: 20 },
  { threshold: 400, points: 25 },
  { threshold: 500, points: 30 },
];
```

```
// Oscar weights (win / nomination)
export const OSCAR_WEIGHTS = {
  best_picture:      { win: 150, nom: 50 },
  director:          { win: 100, nom: 35 },
  lead_actor:         { win: 100, nom: 35 },
  lead_actress:       { win: 100, nom: 35 },
  supporting_actor:    { win: 75,  nom: 25 },
  supporting_actress:  { win: 75,  nom: 25 },
  original_screenplay: { win: 75,  nom: 25 },
  adapted_screenplay: { win: 75,  nom: 25 },
  // below-the-line — apply to any of the remaining categories
  below_the_line:     { win: 20,  nom: 5  },
};

// Other ceremonies score at 30% of Oscar weights
export const OTHER_CEREMONY_MULTIPLIER = 0.3;
export const ELIGIBLE_CEREMONIES = ['golden_globes', 'bafta', 'sag', 'venice', 'b
// Note: Cannes is excluded for the inaugural 2026 season

export const SEQUEL_BO_MULTIPLIER = 0.75;
export const ORIGINAL_IP_AWARDS_MULTIPLIER = 1.25;

export const SLEEPER_BONUS = { budgetCap: 40, grossThreshold: 100, points: 20 };
export const OPENING_WEEKEND_NUMBER_ONE = 5;
```

The engine exposes:

- `calculateBoxOfficePoints(film, currentDomesticGross)` — returns a list of tier-crossing events that have NOT yet been recorded for this film
- `calculateAwardsPoints(film, ceremony, category, isWin)` — returns the points after multipliers
- `calculateSleeperBonus(film, currentDomesticGross)` — returns 20 if eligible and not already awarded, else 0
- `applyMultipliers(film, rawPoints, eventType)` — handles sequel tax and original-IP bonus

The engine never writes to the DB itself; it returns proposed events that the API routes commit.

Inaugural-season rules baked into seed data

Seed the DB with a `LeagueConfig` row (singleton) capturing the 2026-specific rules:

```
{
  seasonYear: 2026,
  draftDate: '2026-06-XX', // fill in
  excludePreDraftReleases: true, // films with releaseDate < draftDate are not e
  excludedCeremonies: ['cannes'],
  rosterSize: 8,
  starterCount: 6,
  budgetCap: 200,
  sequelCap: 3,
}
```

The `/draft` page reads this config and warns if you try to add a film with a release date before the draft date.

Anthropic API integration details

The `/scoring` route uses the API like so:

```
const response = await anthropic.messages.create({
  model: "claude-sonnet-4-20250514",
  max_tokens: 4096,
  system: buildScoringSystemPrompt(currentRoster, leagueConfig),
  tools: [{ type: "web_fetch_20250910", name: "web_fetch" }],
  messages: [
    { role: "user", content: userMessage } // includes URL + instruction
  ],
});
```

The system prompt should:

- List every roster film with manager name and sequel flag
- Include the full scoring rubric inline
- Instruct Claude to fetch the URL, extract relevant data, and return ONLY a JSON code block matching the schema below — no prose outside the code block
- Tell Claude to set `confidence: "low"` for any film it's unsure about

Expected JSON schema in the response:

```
{
  "proposed_events": [
    {
      "film_title": "string (must match roster)",
```

```

    "category": "box_office_tier_50M | box_office_tier_100M | ... | oscar_nom_b
    "raw_points": 10,
    "multiplier": 1.0,
    "reasoning": "what the source said and how it maps",
    "confidence": "high | medium | low"
  }
],
"unmatched_observations": [
  "Any data Claude saw but couldn't map to a roster film"
]
}

```

The server parses this, validates film titles against the roster, and renders the proposal UI. Apply only after explicit confirmation.

UI polish notes

- Leaderboard updates should feel like sports: a small "+15" green flash next to a manager's row when a new score event is added, fading after a few seconds
- Use a deliberately filmic visual style — black background with warm amber accents (#D97706 to match the league doc), serif display font for film titles, mono for numbers
- Every dollar amount and point total should be tabular-figures so columns align
- The /scoring chat should feel like a tool, not a chatbot: render proposals as forms, not paragraphs

Out of scope (for now)

- Real-time multi-user editing — single-admin model is fine
- Email/SMS notifications
- A trade UI beyond the manual /admin route
- Mobile-optimized views — desktop-first
- Historical season data (this is a one-season app; archive at year end and start fresh)

Suggested build order

1. Set up Next.js, Prisma schema, shared-password middleware, basic shadcn layout
2. Build the data model and seed script with the 2026 config and managers

3. Implement `/draft` (bulk paste mode is enough for v1)
4. Implement `/team/[id]` and `/` dashboard with manual scoring (skip Claude integration)
5. Build the scoring engine module with full unit tests against worked examples (e.g., a \$250M sequel film should yield $30 \times 0.75 = 22.5$ points)
6. Wire up `/scoring` with the Anthropic SDK and web fetch
7. Add `/admin` overrides
8. Polish styling

Key worked examples for testing

- Original-IP film grosses \$250M $\rightarrow 5 + 10 + 15 = 30$ box office points, no multiplier
- Sequel film grosses \$250M $\rightarrow 30 \times 0.75 = 22.5$ box office points
- Original-IP film gets nominated for Best Picture $\rightarrow 50 \times 1.25 = 62.5$ awards points
- Original-IP film wins Best Picture (already nominated) $\rightarrow$ additional $(150 - 50) \times 1.25 = 125$ awards points (only the win delta)
- Sub-\$40M budget original film crosses \$100M $\rightarrow 5 + 10 = 15$ BO points + 20 sleeper bonus = 35
- Same film at the Golden Globes wins Best Drama $\rightarrow 150 \times 0.3 \times 1.25 = 56.25$ awards points